

Code Explanation

Data Transformation Functions in PySpark

This code provides a collection of data transformation functions implemented in PySpark. These functions are designed to perform various data processing tasks such as data cleaning, formatting, and preparation for further analysis or storage.

Overview

The code includes a set of functions that can be used to transform and process data stored in DataFrames. These functions cover a range of tasks including data validation, data formatting, and data preparation for output or staging table updates.

Step 1: Checking Source Record Count

The first step involves checking the count of source records in a given DataFrame.

```
def check_source_record_count(df: DataFrame) -> int:
    return df.count()
```

This function takes a DataFrame as input and returns the count of records in it.

Step 2: Formatting Reception Date

The next step is to format the reception date according to the required format.

```
def format_reception_date(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "RECEPTION_DATE",
        to_date(
            to_timestamp(
                col("T_TX_REQUEST_REQ_ACC_DATETIME"),
                "yyyy/MM/dd HH:mm"
            ),
            "yyyy-MM-dd HH:mm"
        )
    )
```

This function takes a DataFrame as input, converts the `T\_TX\_REQUEST\_REQ\_ACC\_DATETIME` column to a timestamp, and then formats it into the desired date format.

Step 3: Normalizing Postal Code

The third step involves normalizing the postal code by removing hyphens and applying fallback logic.

```
def normalize_postal_code(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "NEW_ADDRESS_POSTAL_CODE",
        when(
            (col("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull()) |
            (trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP")) == "") |
            (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))) == 0),
            regexp_replace(col("PPAY_ZIP"), "-", "")
        ).otherwise(
            regexp_replace(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"), "-",
""))
    )
```

This function checks if the `T\_TX\_DTL\_ADD\_CH\_N\_TRANS\_ZIP` column is null or empty, and if so, uses the `PPAY\_ZIP` column as a fallback. It then removes hyphens from the selected postal code.

Step 4: Applying Address Fallback Logic

The fourth step involves applying fallback logic for address fields.

```
def apply_address_fallback(df: DataFrame) -> DataFrame:
    zip_missing = (
        (col("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull()) |
        (trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP")) == "") |
        (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))) == 0)
    )

    return df
        .withColumn(
            "NEW_ADDRESS_KANA_1",
            when(zip_missing, col("PPAY_ADR1_FW"))
            .otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ADD1"))
        )
        .withColumn(
            "NEW_ADDRESS_KANA_2",
            when(zip_missing, col("PPAY_ADR2_FW"))
            .otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ADD2"))
        )
        .withColumn(
            "NEW_ADDRESS_KANA_3",
            when(zip_missing, col("PPAY_ADR3_FW"))
            .otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ADD3"))
        )
        .withColumn(
            "NEW_ADDRESS_KANJI_1",
            col("T_TX_DTL_ADD_CH_N_TRANS_AD1_KJ")
        )
        .withColumn(
            "NEW_ADDRESS_KANJI_2",
            col("T_TX_DTL_ADD_CH_N_TRANS_AD2_KJ")
        )
        .withColumn(
            "NEW_ADDRESS_KANJI_3",
            col("T_TX_DTL_ADD_CH_N_TRANS_AD3_KJ")
        )
    )
```

This function checks if the `T\_TX\_DTL\_ADD\_CH\_N\_TRANS\_ZIP` column is null or empty, and if so, uses the corresponding `PPAY\_ADR` columns as a fallback for the address fields.

Step 5: Formatting Telephone Number

The fifth step involves formatting the telephone number according to the required format.

```
def format_telephone_number(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "NEW_ADDRESS_TELEPHONE_NUMBER",
        when(
            (col("T_TX_DTL_ADD_CH_N_TRANS_PHNO").isNull()) |
            (trim(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO")) == "") |
            lit("")
        ).otherwise(
            when(
```

```

        substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 1, 3)
.isin(
        "050", "060", "070", "080", "090"
),
        concat(
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 1
, 3),
            lit("-"),
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 4
, 4),
            lit("-"),
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 8
, 4)
        )
).otherwise(
        concat(
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 1
, 2),
            lit("-"),
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 3
, 4),
            lit("-"),
            substring(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"), 7
, 4)
        )
    )
)
)
)
)

```

This function formats the telephone number by checking its prefix and then formatting it accordingly.

Step 6: Formatting Policy Owner Name

The sixth step involves formatting the policy owner name from the lookup data.

```
def format_policy_owner_name(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "POLICY_OWNER_NAME_KANA",
        when(
            (col("POWN_FNM").isNull()) | (trim(col("POWN_FNM")) == "
"),
            trim(col("POWN_LNM"))
        ).otherwise(
            concat_ws(" ", trim(col("POWN_LNM")), trim(col("POWN_FNM
"))))
    )
    ).withColumn(
        "POLICY_OWNER_NAME_KANJI",
        col("POWN_KNM")
    )
```

This function formats the policy owner name by concatenating the last name and first name, and also retrieves the Kanji name.

Step 7: Adding Sequence Number

The seventh step involves adding a sequence number to the DataFrame.

```
def add_sequence_number(df: DataFrame) -> DataFrame:
    return df.withColumn("SEQUENCE_NUMBER", monotonically_increasing_id())
```

This function generates a unique sequence number for each row in the DataFrame.

Step 8: Preparing Staging Update

The eighth step involves preparing the DataFrame for staging table update with audit fields.

```
def prepare_staging_update(df: DataFrame, process_userid: str) -> DataFrame:
    return df.withColumn(
        "STG_PROCESS_DATE",
        current_timestamp()
    ).withColumn(
        "STG_PROCESS_USERID",
        lit(process_userid)
    )
```

This function adds two audit columns, `STG\_PROCESS\_DATE` and `STG\_PROCESS\_USERID`, to the DataFrame.

Step 9: Selecting Output Columns

The ninth step involves selecting the columns for the output file.

```
def select_output_columns(df: DataFrame) -> DataFrame:
    return df.select(
        "SEQUENCE_NUMBER",
        "ORIGIN_REQUEST_ID",
        "POLICY_ID",
        "RECEPTION_DATE",
        "NEW_ADDRESS_POSTAL_CODE",
        "NEW_ADDRESS_KANA_1",
        "NEW_ADDRESS_KANJI_1",
        "NEW_ADDRESS_KANA_2",
        "NEW_ADDRESS_KANJI_2",
        "NEW_ADDRESS_KANA_3",
        "NEW_ADDRESS_KANJI_3",
        "NEW_ADDRESS_TELEPHONE_NUMBER",
        "POLICY_OWNER_NAME_KANA",
        "POLICY_OWNER_NAME_KANJI"
    )
```

This function selects the required columns for the output file.

Step 10: Selecting Staging Update Columns

The final step involves selecting the columns for staging table update.

```
def select_staging_update_columns(df: DataFrame) -> DataFrame:
    return df.select(
        "T_TX_REQUEST_REQUEST_ID",
        "T_TX_BASIC_TRANS_ID",
        "T_TX_RELATION_TRANS_REL_ID",
        "T_TX_REQ_POL_REQUEST_POLICY_ID",
        "STG_POLICY_ID",
        "STG_TXDB_STATUS",
        "STG_PROCESS_DATE",
        "STG_PROCESS_USERID"
```

```
)
```

This function selects the required columns for the staging table update.